

Navis 횡단보도 시연 가이드

연동 점검 결과 · 시연 시나리오 · 리허설 체크리스트 · 예상 질문(기술/안전성)

1. 세 코드 연동 점검 결과

Raspberry Pi, ESP32, Flutter 앱이 주고받는 메시지 이름을 스크립트로 대조했습니다. 한쪽이 보내는 메시지는 모두 받는 쪽에 처리 코드가 있습니다. 메시지 규칙은 GitHub의 docs/PROTOCOL.md 한 문서로 통일했습니다.

방향	메시지	결과
Raspberry Pi → ESP32	CROSSWALK:0/1, LIGHT:RED, LIGHT:GREEN, LIGHT:GREEN_WAIT, CROSSING_START/END, CROSS_MOTOR:L/R/CENTER (0.3초마다 반복)	모두 처리됨
ESP32 → 앱	DIST, SIDE, OBSTACLE, OBSTACLE_BRAKE, AVOID:L/R/BLOCKED, OBSTACLE_CLEAR, 횡단보도 · 신호 메시지 전달	모두 처리됨
앱 → ESP32	B:방위각, L/R/F:각도, S:0	모두 처리됨

주의: 메시지 이름과 흐름이 맞는다는 뜻이며, 실제 보드와 카메라로는 아직 돌려보지 않았습니다. ESP32 코드는 PC에서 가짜 센서·가짜 Pi 메시지로 시험했고, Pi 코드는 전송 로직만 따로 시험했습니다. 반드시 아래 리허설 체크를 한 번 거쳐 주세요.

2. 시연 준비와 배치

2.1 추천 방식

횡단보도 완전히 옆에서 출발하는 것은 추천하지 않습니다. 카메라 앞쪽에 횡단보도가 보여야(박스 아래쪽이 화면 40% 선에 닿아야) 판단을 시작하기 때문입니다. 실제 시각장애인도 연석 앞에서 건너편을 향해 출발합니다.

대신 한 번 건너는 동안 다음 두 장면을 모두 보여줍니다.

- **중앙 유도:** 횡단보도 한쪽 가장자리에서 출발 → 지팡이가 중앙 쪽으로 당김
- **방향 복귀:** 중간에 시연자가 일부러 지팡이를 비틀 → 다시 직진으로 바로잡음

2.2 배치

항목	준비 내용
조명	밝고 균일하게. 역광(창문을 등진 신호등)과 강한 그림자는 인식을 흔들
신호등	횡단보도 건너편 끝, 카메라 정면 약간 위. 출발 위치에서 HUD 화면 안에 들어오는지 확인

출발 위치	횡단보도 앞 끝, 중앙에서 40~50 cm 옆. 몸은 건너편을 향하고 지팡이는 5~10° 살짝 틀어 둠
신호 상태	출발 전 반드시 빨간불. 빨간불 → 초록불로 바뀌는 것을 봐야 출발함
장애물	상자 하나를 도착 지점 뒤쪽이나 별도 구간에 배치 (횡단보도 한가운데는 피함)
스마트폰	지팡이에 단단히 고정, 볼륨 최대. 시끄러우면 블루투스 스피커 사용
모니터	Pi HUD 화면을 심사위원 쪽에 배치 (박스, 신호 색, off 값이 보임)

3. 시연 진행 순서 (약 1분 30초)

시간	진행	보여지는 동작 / 음성
10초	문제 소개	"음향신호기가 없는 횡단보도가 많고, 건너는 중 방향이 틀어지기 쉽습니다."
15초	안대를 쓴 시연자가 출발 위치에 섬 (빨간불)	"앞에 횡단보도가 있습니다" → "빨간불입니다. 정지해 주세요" (지팡이 정지)
5초	신호를 초록불로 바꿈	"초록불입니다. 건너세요"
10초	한쪽 끝에서 출발	지팡이가 중앙 쪽으로 당김. HUD의 off 값이 0에 가까워짐
10초	중간에서 일부러 지팡이를 비튼	반대쪽으로 당겨 직진 복귀
5초	도착	"횡단보도를 다 건넜습니다"
20초	상자 앞으로 걸어감	역토크 3회 → "장애물이 감지되었습니다" → "오른쪽으로 피해 주세요" → 트일 때까지 당김 → "장애물을 지났습니다"
선택	처음부터 초록불인 상태로 도착	"초록불이 이미 켜져 있습니다. 다음 초록불에 건너겠습니다" (안전 설계 강조)

건너는 도중 신호등: 바꾸지 않아도 됩니다. Pi는 건너기 시작하면 신호등을 보지 않고 횡단보도 방향만 봅니다.

4. 리허설 체크리스트 (시연 전날)

- ☐ **좌우 방향:** 횡단보도를 카메라 왼쪽에 두면 HUD의 off가 음수, 지팡이는 왼쪽으로 당겨야 정상. 반대면 main.py의 FLIP_LEFT_RIGHT = False
- ☐ **모델 파일:** best2_ncnn_model 폴더가 Pi에 있는지 (GitHub에는 없음)
- ☐ **FPS:** 3 FPS 이하면 신호 인식에 3초 이상 걸림 → main.py의 10프레임 기준을 5로
- ☐ **도착 판정:** "다 건넜습니다"가 너무 늦거나 빠르면 missing_cw_frames >= 20의 20 조정
- ☐ **당기는 세기:** 안대를 쓴 사람이 확실히 느끼는지. 약하면 CROSS_PWM 60→70, AVOID_PWM 70→80
- ☐ **통신 끊김:** Pi 케이블을 뽑으면 1초 안에 모터가 멈추는지 (질문 대비 시연용)
- ☐ **업로드:** 앱, ESP32, Pi 세 개 모두 새 버전인지

□ **BLE:** 앱 실행 시 SMART_CANE 자동 연결, "연결됨" 표시 확인

5. 예비 계획

- **Pi 인식이 불안정할 때:** main.py를 다시 실행하고, 카메라 케이블·조명·신호등 위치를 확인. HUD에서 횡단보도 박스와 신호 색이 잡히는지 본 뒤 다시 시작
- **모터가 반대로 당길 때:** main.py의 FLIP_LEFT_RIGHT 값을 바꾸고 다시 실행
- **BLE 연결 실패:** 앱의 "SMART_CANE 연결" 버튼으로 재시도, ESP32 전원 재인가

6. 조정 가능한 값

위치	변수	현재 값	이럴 때 조정
Pi	CENTER_ENTER_RATIO	0.12	중앙 유도가 너무 예민/둔할 때
Pi	ANGLE_ENTER_DEG	3.0°	좌우로 자주 흔들리면 4~5
Pi	FLIP_LEFT_RIGHT	True	당기는 방향이 반대일 때
Pi	CROSS_RESEND_SEC	0.3초	보정 반복 전송 간격
ESP32	CROSS_PWM / CROSS_PULSE_TIME	60 / 300 ms	횡단 보정 힘이 약할 때 ↑
ESP32	CROSS_TIMEOUT_MS	1000 ms	Pi 소식 없을 때 정지까지 시간
ESP32	AVOID_PWM / AVOID_MAX_PULSES	70 / 8회	회피 힘, 길 막힘 판단 횟수
앱	headingToleranceDegrees	25°	길안내가 아직 예민하면 ↑

7. 예상 질문 - 기술

Q1. 진동이나 음성이 아니라 왜 모터로 방향을 알려주나요?

A. 진동·음성은 사용자가 정보를 듣고 다시 해석해야 합니다. 바퀴 토크는 "이쪽으로"를 몸으로 바로 느끼게 해 반응이 빠르고, 시끄러운 도로에서도 전달됩니다. 음성은 보조로 함께 사용합니다.

Q2. 초록불이 이미 켜져 있을 때 도착하면 어떻게 하나요?

A. 빨간불에서 초록불로 바뀌는 순간을 봐야만 출발시킵니다. 이미 초록불이면 남은 시간을 모르므로 "다음 초록불에 건너겠습니다"라고 안내하고 기다립니다.

Q3. 신호등 색은 어떻게 판단하고, 오인식은 어떻게 막나요?

A. YOLO로 신호등 영역을 찾고 HSV로 빨강/초록 픽셀 비율·채도·밝기 점수를 계산합니다. 10프레임 연속 같은 색이어야 바꾸고(히스테리시스), 신호등이 20프레임 안 보이면 이전 값을 무효화합니다.

Q4. 건너는 중 방향은 어떻게 판단하나요?

A. 두 단계입니다. 횡단보도 박스가 화면 한쪽으로 12% 이상 치우치면 먼저 중앙 쪽으로 보냅니다. 중앙 근처에서는 줄무늬를 Canny 에지와 Hough 직선으로 찾아 기울기를 구하고, EMA로 흔들림을 줄여 좌/우/직진을 정합니다.

Q5. 일반 길안내는 GPS 오차가 큰데 괜찮나요?

A. 스마트폰 GPS는 보통 3~5 m 오차라서, 경로선 좌우 5 m(정확도가 나쁘면 최대 10 m)까지는 경로 위로 보고 경로와 나란히 건도록만 안내합니다. 나침반은 0.7초 평활화와 $\pm 25^\circ$ 허용 범위를 두었고, Android 나침반의 자북과 지도 진북의 차이(한국 약 9°)도 보정했습니다.

Q6. 장애물이 앞과 옆을 다 막으면요?

A. 좌우 VL53L1X 센서로 트인 쪽을 찾아 정면이 트일 때까지 계속 유도합니다. 양쪽 다 막혀 있거나 8번 당겨도 안 트이면 역토크로 멈춤을 알리고 "앞이 막혀 있습니다"라고 안내합니다. 양쪽이 비슷하면 우측 보행 원칙에 따라 오른쪽으로 갑니다.

Q7. 세 장치는 어떻게 역할을 나누나요?

A. Raspberry Pi는 눈(영상 인식), 스마트폰은 길안내·음성·화면, ESP32는 모든 입력을 모아 모터를 제어하는 중심입니다. 메시지 규칙은 docs/PROTOCOL.md 한 문서로 통일했습니다.

Q8. 기존 제품(스마트 지팡이, 음향신호기)과 차이는요?

A. 기존 제품은 "알려주는" 데 그칩니다. Navis는 인식 → 판단 → 방향 유도까지 한 시스템으로 묶었고, 특히 횡단 중 방향 유지와 중앙 유도까지 합니다.

Q9. 실제 도로에서의 한계는 무엇인가요?

A. 솔직하게 답하는 것이 좋습니다.

- 야간, 역광, 비 오는 날의 인식을
- 폭이 넓은 실제 횡단보도에서의 카메라 시야
- Raspberry Pi 4의 처리 속도(FPS)
- 스마트폰 나침반이 모터·금속에 받는 자기 간섭

개선 방향: 교통신호 정보 API/V2X 연동, 경량 모델과 가속기, IMU 기반 방향 추정

Q10. 모델은 어떻게 학습했고 정확도는요?

A. (팀에서 준비) 데이터 수, 클래스(cross-walk, traffic_light, ground_light 등), mAP, Pi에서의 FPS를 한 장으로 정리해 두세요.

8. 예상 질문 - 안전성

보조기기는 "잘 될 때"보다 "잘못될 때 어떻게 되는가"를 많이 묻습니다. 답변은 [현재 대책] + [남은 한계와 개선 계획] 순서로 하면 신뢰를 얻기 좋습니다.

Q1. 모터가 사람을 끌고 가다가 넘어지게 하지 않나요?

A. 모터는 끌고 가는 장치가 아니라 방향 신호입니다. 사용자가 밀고 가는 지팡이에 짧게(0.3~0.4초) 약한 힘(PWM 55~70/255, 약 22~27%)을 줄 뿐이고, 사용자가 손을 놓거나 버티면 그대로 멈춥니다. 최종 판단은 항상 사용자가 합니다.

Q2. 모터가 오작동해서 계속 돌면요?

A. 모든 유도 동작은 "끝나는 시각"이 정해진 짧은 펄스라서 스스로 꺼집니다. 추가로:

- Pi 메시지가 1초 끊기면 ESP32가 횡단 보정을 멈춤
- 앱 BLE 연결이 끊기면 ESP32가 즉시 모터 정지

- 빨간불·신호 대기 중에는 보정 명령이 와도 모터를 돌리지 않음

개선 계획: 물리적 비상정지 버튼, 모터 전류 감시

Q3. 카메라가 신호를 잘못 봐서 빨간불에 건너라고 하면요?

A. 가장 위험한 경우라 여러 겹으로 막았습니다.

- 한 프레임이 아니라 10프레임 연속 같은 색이어야 인정
- 빨간불을 먼저 보고 초록불로 바뀌어야만 출발 (초록불 중간 도착 시 대기)
- 신호등이 안 보이면 이전 색을 버림

한계: 조명이나 비슷한 색 광고판에 영향을 받을 수 있습니다. 실사용에서는 교통신호 개방 데이터/V2X와 교차 확인하고, 음향신호기가 있으면 그것을 우선하는 것이 목표입니다.

Q4. 건너는 도중 신호가 바뀌면요?

A. 한가운데서 멈추는 것이 더 위험하므로, 건너기 시작한 뒤에는 멈추게 하지 않고 방향 보정을 계속해 끝까지 건너게 합니다. 출발 조건을 "빨간불 → 초록불로 바뀌는 순간"으로 제한한 것도 초록불 시간을 최대한 확보하기 위해서입니다. 개선 계획: 건너는 중에도 신호를 감시해 신호가 바뀌면 "서둘러 건너세요"라고 안내.

Q5. 방향 보정이 틀리면 차도로 유도하지 않나요?

A. 보정은 횡단보도 박스 안에서만 동작합니다. 횡단보도가 20프레임 동안 안 보이면 보정을 끝냅니다. 좌우 판단에는 히스테리시스와 EMA를 두어 한두 프레임의 오인식으로 방향이 바뀌지 않게 했습니다. 한계: 차량은 인식하지 않으므로 차량 감지는 향후 과제입니다.

Q6. 장애물 센서가 못 보는 장애물은요?

A. 정면 TFmini는 한 점을 재는 센서라 낮은 턱, 구덩이, 계단 아래, 머리 높이의 돌출물은 놓칠 수 있습니다. 그래서 Navis는 흰지팡이를 대체하는 것이 아니라 보조하는 장치이며, 사용자의 기존 보행 기술을 전제로 합니다. 개선 계획: 하향 거리 센서, 카메라 기반 계단·턱 인식(모델에 계단 클래스 포함).

Q7. 장애물을 피하다가 다른 위험(차도, 사람) 쪽으로 가면요?

A. 좌우 거리 센서로 30 cm 안이 막힌 쪽으로는 보내지 않고, 양쪽 다 막히면 멈춥니다. 8번 유도해도 정면이 안 트이면 "길 막힘"으로 멈추고 음성으로 알립니다. 무조건 피하게 하기보다 멈추는 쪽을 택했습니다.

Q8. 스마트폰 앱이 꺼지거나 배터리가 떨어지면요?

A. 장애물 감지와 역토크 경고는 ESP32가 혼자 처리하므로 앱 없이도 동작합니다. 앱 연결이 끊기면 길안내 유도는 멈춥니다. 한계: 현재 지팡이 배터리 잔량 경고가 없어 추가가 필요합니다.

Q9. 여러 장치 명령이 동시에 오면 충돌하지 않나요?

A. 우선순위를 정했습니다: 장애물 회피 > 빨간불 대기 > 횡단 보정(Pi) > 일반 길안내(앱). 장애물 회피 중에는 다른 명령이 모터를 건드리지 못하고, 횡단 중에는 앱의 길안내 명령을 무시합니다.

Q10. 카메라가 사람 얼굴을 찍는데 개인정보 문제는 없나요?

A. 영상은 Raspberry Pi 안에서만 처리하고 저장하거나 외부로 전송하지 않습니다. ESP32로는 "CROSS_MOTOR:L" 같은 판단 결과만 보냅니다.

Q11. 실제 시각장애인에게 시험해 봤나요?

A. (팀 상황에 맞게) 현재는 안대를 쓴 팀원과 모의 횡단보도로 시험했습니다. 실제 사용자 평가는 기관 협력과 안전 절차를 갖춰 진행할 계획이라고 솔직히 답하는 것이 좋습니다.